

Dokumentation

Duck-Duck-Düsseldorf

Die verwendeten Globalen Signale und ihre Verknüpfungen
Allgemeine Abhängigkeiten des Codes

Inhaltsverzeichnis

Einführung	2
Grundidee des Spieles und Entwicklungsablauf	2
Nutzung der Gruppenfunktion in Godot	2
Benutzte Layer und Masken Ebenen	3
Signale – Verknüpfungen und Auslöser – Tabellen	4
Allgemeine Signale.....	5
Sound Signale	7
Dialog Signale.....	8
Timer Signale.....	9
Visuelle Darstellungen der Beziehungen	10
Gesamtübersicht.....	10
Scene Controller	11
Level Manager.....	12
Item Manager	12
Globale Signale Übersicht	13
Globale Sounds Übersicht.....	14
Boss State Machine Übersicht	15
Nachtrag.....	16
Bekannte Bugs.....	16
Potentielle Erweiterungen für die Zukunft.....	16

Einführung

Grundidee des Spieles und Entwicklungsablauf

Das Spielprinzip von Duck-Duck-Düsseldorf basiert auf das Spiel [Vampire Survivors](#) (ein Action-Rougelike Spiel).

Vor Beginn der Umsetzung hat man sich auf einen Game-Loop und eine Story geeinigt, was dann in verschiedene Aufgaben-/Themenbereiche aufgeteilt wurde.

Als nächstes hat man damit begonnen, sich auf die Grundelemente des Spiels zu konzentrieren, die für den Game-Loop essenziell sind. Darunter fallen zum Beispiel Spieler und Gegner.

Jeder Bereich hat seine speziellen Funktionen, mit denen man sich als Team auseinandergesetzt hat. Allerdings werden manche Funktionen und Variablen auch bereichsübergreifend verwendet, weswegen man sich auf bestimmte Syntaxen geeinigt hat, um die richtigen Funktionen zu nutzen.

Als Beispiel dafür sind die Methoden, die mit einem “_” beginnen. Daran erkennt man, dass diese nur für die Scene selbst relevant sind und nicht für die äußere Verwendung zu benutzen sind.

Nutzung der Gruppenfunktion in Godot

Gruppen wurden gezielt eingesetzt, um Objekte wie Spieler, Gegner oder sammelbare Items zuverlässig identifizieren und verarbeiten zu können. Die Methode `is_in_group(...)` erlaubte dabei eine flexible und saubere Abfrage, unabhängig von der konkreten Szene oder Vererbung des Objekts.

Gruppe: Player

Diese Gruppe kennzeichnet den Spielercharakter. Sie wird an vielen Stellen verwendet, z. B. bei Gegnern oder Triggerzonen, um zu prüfen, ob der Spieler interagiert.

Gruppe: Drops

Alle sammelbaren Objekte wie `drop_energydrink`, `duck_collectable` oder `drop_chocolatebar` gehören zu dieser Gruppe. Der Spieler prüft bei Kollision, ob das berührte Objekt ein erlaubtes Sammelobjekt ist.

Benutzte Layer und Masken Ebenen

Damit ein Spieler mit seiner Umgebung interagieren kann, ist es besonders hilfreich die Map in mehreren Schichten zu unterteilen. So sind bestimmte Funktionen auf einzelne Ebenen, ohne das andere Elemente beeinflusst werden.

Layer: TileMapLayer

Das sind die Basiselemente einer Map. Hier sind die Designs eingebettet und mit verschiedenen Masken kombiniert (siehe nächste Seite).

Layer: Area2D + CollisionShape2D

Bei den Area2D Ebenen handelt es sich um zusätzliche Layer, die als Childnode eine CollisionShape gesetzt bekommen. Wenn es zu einer Kollision zwischen dem Spieler und der Shape2D kommt, löst das Area2D ein Signal aus, was dann ein Script ausführen kann.

Masken: Navigation Layers

Hierbei handelt es um Einstellungen, die innerhalb eines TileMapLayers unter *Tile Set* vorgenommen werden. Dadurch können spezifische Bereiche auf der Ebene ausgewählt werden, die zum Beispiel für die Navigation von Gegnern relevant sind.

Masken: Physics Layers

Diese werden ähnlich zu den Navigation Layern innerhalb eines TileMapLayers eingestellt. So hat man die Möglichkeit, bestimmte Bereiche in einem TileSet eine Kollisionseigenschaft zu geben.

Beispielsweise damit ein Spieler nicht durch Wände laufen kann.

Signale – Verknüpfungen und Auslöser – Tabellen

In den Skripten werden Signale verwendet, um Szenen übergreifend zu kommunizieren. Die definierten globalen Signale findet man im Skript "global_signals.gd", welches bei Programmstart automatisch geladen wird, es ist also ein globales Skript.

Die folgenden Tabellen sind per Kategorie in zwei Versionen aufgeteilt.

Zuerst wird die Ansammlung aller auslösenden Signale per Skript sortiert aufgelistet.

GlobalSignals.Signalname.emit(eventuelle Parameter)

Skript	Sendet Signal (emit)	In der Funktion
Das Skript in denen das Signal ausgelöst wird.	Das ausgelöste Signal. Die Angaben in den Klammern sind Parameter, die übergeben werden.	Welche Funktion im Skript das Signal auslöst.

Danach folgt die Ansammlung aller Signale und deren Verknüpfungen in den einzelnen Skripten. Die Verknüpfung an sich ist in den jeweiligen _ready() Funktionen der Skripte angegeben.

GlobalSignals.Signalname.connect(Funktionsname)

Signal	Ist verbunden mit Funktion	Im Skript
Das Signal (was theoretisch ausgelöst wurde)	Die Funktion, welche beim auslösen des Signals aufgerufen wird. Die Angaben in den Klammern sind Parameter, die die Funktion erhält.	In welchem Skript die Funktion steht.

Der Inhalt der Tabellen ist nach Typ des Objektes sortiert:

Manager – Player – nonPlayer – Interactables – Environment - HUD

Allgemeine Signale

Dies ist eine Ansammlung aller nicht kategorisierten Signale. Skript individuelle Signale werden nicht aufgeführt.

game_won steht hierbei dafür, dass genug Enten eingesammelt wurden. Das wirkliche Ende des Games wird durch boss_slain gehandelt.

Skript	Sendet Signal (emit)	In der Funktion
level_manager	game_won	[ALL] duck_collected_counter()
	duck_collected_levelUp	[LEVELUP] duck_collected_counter()
player	game_over	_on_animation_player_animation_finished("scale")
	reduce_mob_counter	_on_despawnrange_body_exited()
mob	drop_duck (global_position)	drop_item()
	bossMinion_item (global_position)	liberated()
	reduce_mob_counter	_on_time_to_live_timeout()
ranged_mob	drop_duck (global_position)	drop_item()
	bossMinion_item (global_position)	liberated()
	reduce_mob_counter	_on_time_to_live_timeout()
ent_boss	drop_duck (global_position)	drop_item()
	phase2_reached()	take_damage()
	boss_slain	_die()
destroyable_trashcan	drop_item (global_position)	take_damage()
destroyable_backpack	drop_item (global_position)	take_damage()
duck_collectable	duck_collected_signal	_on_collection_area_body_entered()
elevator_portal	toggle_mob_drops toggle_natural_spawns	_on_body_entered()

Signal	Ist verbunden mit Funktion	Im Skript
reduce_mob_counter	reduce_mob_counter()	level_1
toggle_natural_spawns	toggle_spawn_cycle()	level_1
duck_collected_signal	duck_collected_func()	ingame
	duck_collected_counter (amount)	level_manager
duck_collected_levelUp	_levelUp()	player player_weapon mob ranged_mob mob_projectile
toggle_mob_drops	boss_fight_started()	ranged_mob mob
game_won	_on_game_won()	player
	_reset_mob_lvl()	ranged_mob mob
	show_elevator()	elevator_portal
game_over	_game_over()	level_manager
	_reset_mob_lvl()	ranged_mob mob
drop_duck	_drop_duck (global_position)	item_manager
drop_item	_drop_item (global_position)	item_manager
bossMinion_item	_drop_bossMinion_item (global_position)	item_manager
phase2_reached	phase2Started()	LaserAttackState MovingState TeleportState
boss_slain	duck_reset()	level_manager
	_set_endScene_flag()	fade_on_roof

Sound Signale

Hier sind alle Signale zu finden, die den Sound global steuern. Szenen individuelle Sounds sind hiervon ausgenommen.

Die globalen Sounds findet man im "scene_controller.tscn" in der Sounds Node, akzeptierte Soundnamen sind somit nur "level_sound", "typing_sound" und "starting_sound", welche beim auslösen des Signals als String mitgegeben werden.

Skript	Sendet Signal - Sounds (emit)	In der Funktion
level_1	play_sound ("level_sound")	_ready()
dialogue_ende	play_sound ("typing_sound")	_on_timer_timeout()
	stop_sound ("typing_sound")	_input() _on_timer_timeout()
dialogue_begin	play_sound ("typing_sound")	_on_timer_timeout()
	stop_sound ("typing_sound")	_input() _on_timer_timeout()
startscreen	play_sound ("starting_sound")	_on_start_button_pressed()
game_over	play_sound ("starting_sound")	_on_menu_pressed() _on_quit_pressed()
player	stop_sound ("level_sound")	_die()

Signal	Ist verbunden mit Funktion	Im Skript
play_sound	_play_sound (sound_name)	sounds
stop_sound	_stop_sound (sound_name)	sounds

Dialog Signale

Diese Signale triggern Dialog Sequenzen oder signalisieren das Ende eines Dialogs. Um ein Dialog zu starten, muss eine JSON aus den Assets (mitsamt Pfad!) mitgegeben werden, welche den Text für die Szene enthält. Während eine Dialogszene läuft und eine Game Szene im Scene Controller aktiv ist, wird diese zudem über den Process Mode pausiert.

```
GlobalSignals.dialogue_start.emit("res://CutScene/Dialogue/ZwischenDialog_2.json")
```

Die [eckigen Klammern] deuten hier auf die Auslösebedingung im Code selber hin.

[ONCE] – Wird nur einmal im Gameablauf ausgelöst

[AT HALF] – Wenn man die Hälfte der benötigten Enten eingesammelt hat

[AT ALL] – Wenn man alle benötigten Enten eingesammelt hat

Skript	Sendet Signal - Dialoge (emit)	In der Funktion
dialogue_begin	dialogue_finished	_input() next_script()
player	dialogue_start (dialog_anfang.json)	[ONCE] _process()
	dialogue_start (ZwischenDialog_1.json)	[ONCE] _on_pick_up_area_entered()
level_manager	dialogue_start (ZwischenDialog_2.json)	[AT HALF] duck_collected_counter()
	dialogue_start (ZwischenDialog_3.json)	[AT ALL] duck_collected_counter()

Signal	Ist verbunden mit Funktion	Im Skript
dialogue_start	_pause_level_on_dialogue_start (dialogue_path)	level_1
	start (dialogue_path)	dialogue_begin
dialogue_finished	_unpause_level_on_dialogue_end()	level_1

Timer Signale

Diese globalen Timer regulieren die Effekte, die auf den Spieler wirken. Diese global zu regeln ermöglicht es, dass ein wiederholtes aufsammeln eines (z.B.) Speed Boost zu einer Verlängerung des Effekts führt, statt zu einer Addierung oder zur Wirkungslosigkeit.

Zum starten der Timer verwendet man die start Funktion. Diese verlängert den Effekt automatisch, falls einer gerade schon läuft.

Momentan sind ein Speed Boost und ein Firerate Boost implementiert.

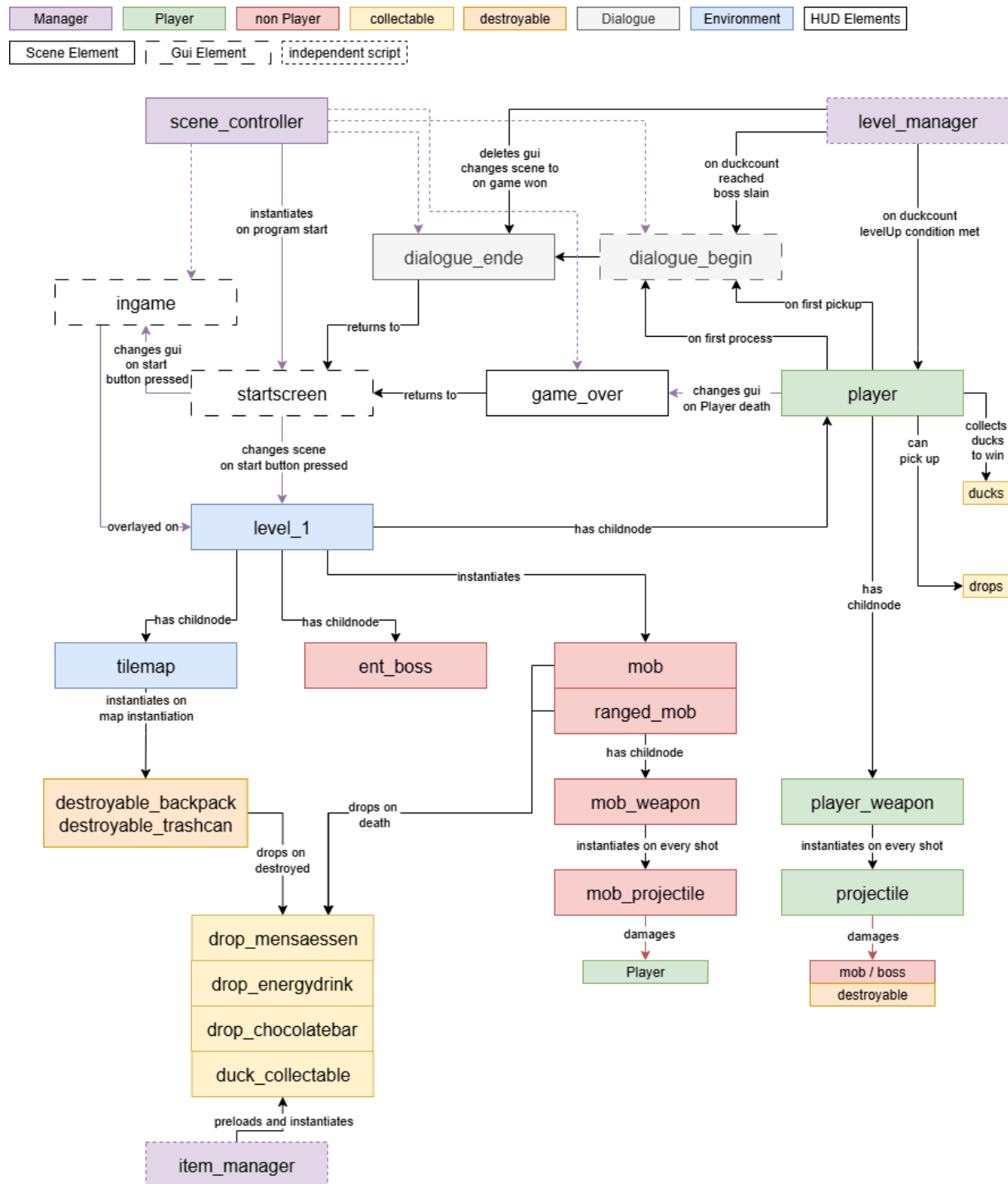
Skript	Sendet Signal – Timer (start)	In der Funktion
drop_energydrink	timerSpeedUp (effect_length)	pickup()
drop_chocolatebar	timerFirerate (effect_length)	pickup()

Signal	Ausgelöst durch	Ist verbunden mit Funktion	Im Skript
timerSpeedUp	"timeout"	_on_global_speedUp_timeout	player
timerFirerate	"timeout"	_on_global_firerate_timeout	player_weapon

Visuelle Darstellungen der Beziehungen

Gesamtübersicht

Dieses Diagramm stellt die allgemeinen Beziehungen der Szenen dar und bietet einen ersten Überblick. Die farbliche Legende teilt die Objekte in grobe Kategorien ein, wobei Manager bereits bei programmstart geladen sind und die Abläufe bestimmen.

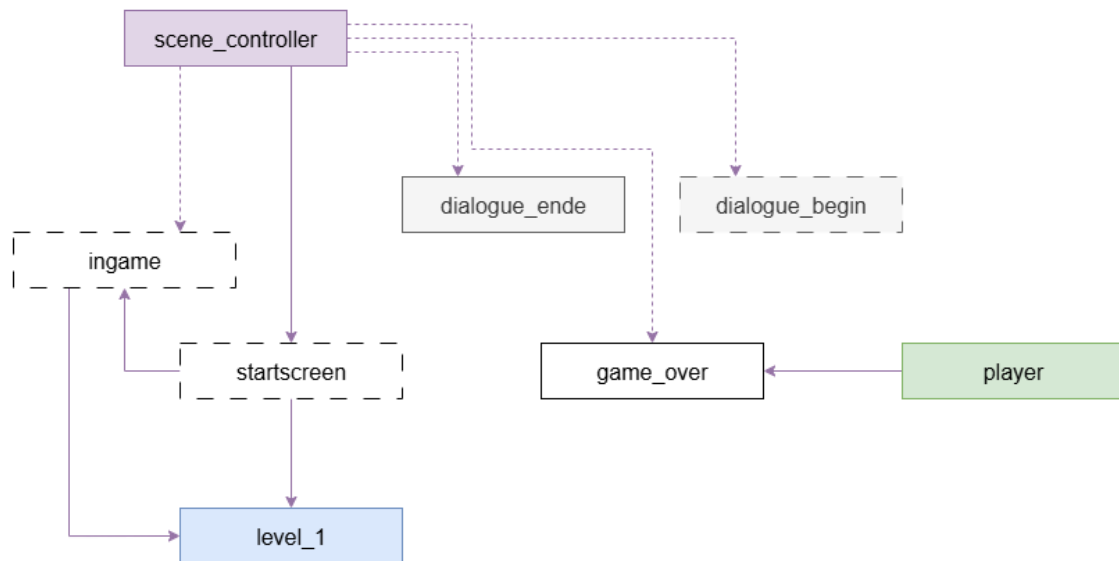


Scene Controller

Dies ist unsere Main Scene und steuert das laden und entfernen von Nodes. Hier wird in Game Szenen und Gui Szenen (HUD) unterschieden, da diese unabhängig voneinander manipulierbar sein müssen.

Beispiel: level_1 ist geladen mit ingame HUD, wenn der Player stirbt, wird die ingame HUD eingefroren und die gameover HUD geladen, während level_1 bestehen bleibt.

Hier ist zudem der globale Sound zu finden, sowie der Sound und Level Manager.



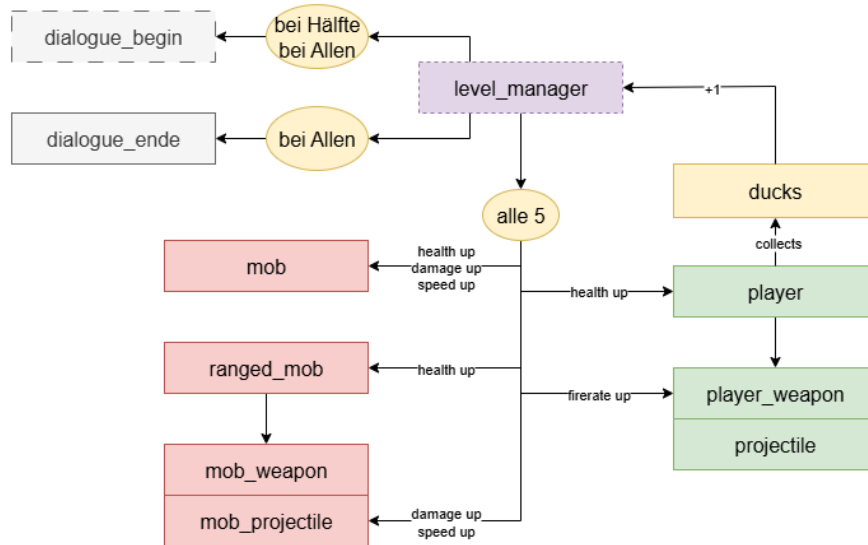
Level Manager

Dieser zählt per Signal, wenn der Player eine Ente einsammelt, die insgesamt gesammelten Enten und überprüft mit jedem Aufruf, ob bestimmte Bedingungen erfüllt wurden.

Genug Enten für ein LevelUp -> Signal an Mobs und Player

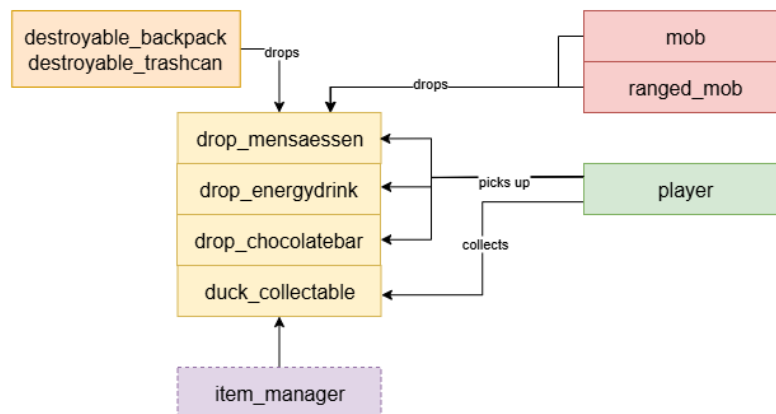
Hälfte der Enten für den Sieg -> Signal für Dialog

Alle Enten für den Sieg -> Signal an Player, Dialog und Map (Aufzug)



Item Manager

Dieses Skript preloads alle Drop Szenen und die Enten Szene, und reagiert auf das Signal drop_item / drop_duck / minionBoss_item. Es beinhaltet unterschiedliche Varianten an Chancen für diese Drops, je nach Gamestate. Beim Boss werden z.B. keine Ducks mehr gedroppt.



Globale Signale Übersicht

Hier findet man alle global relevanten Signale, Klassen Variablen und Timer für globale Effekte. Man kann diese via `GlobalSignals.variable_name` aufrufen und die jeweiligen Funktionen verwenden.

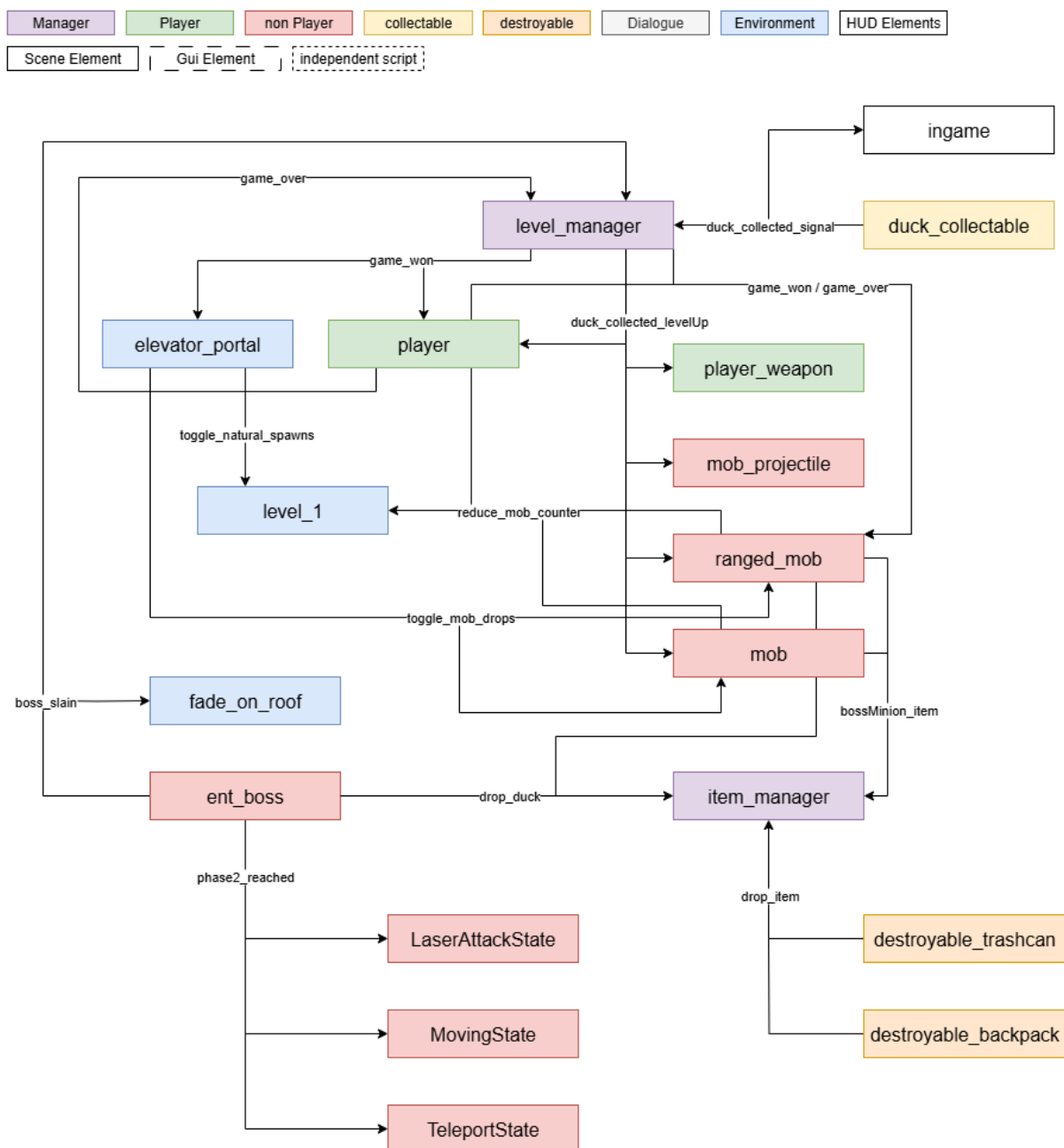
Unter Klassen kann man adressierbare Nodes verstehen, welche einen globalen Zugriff auf die Szene ermöglichen.

Level_Manager = duck_counter

Scene_Controller = scene_controller

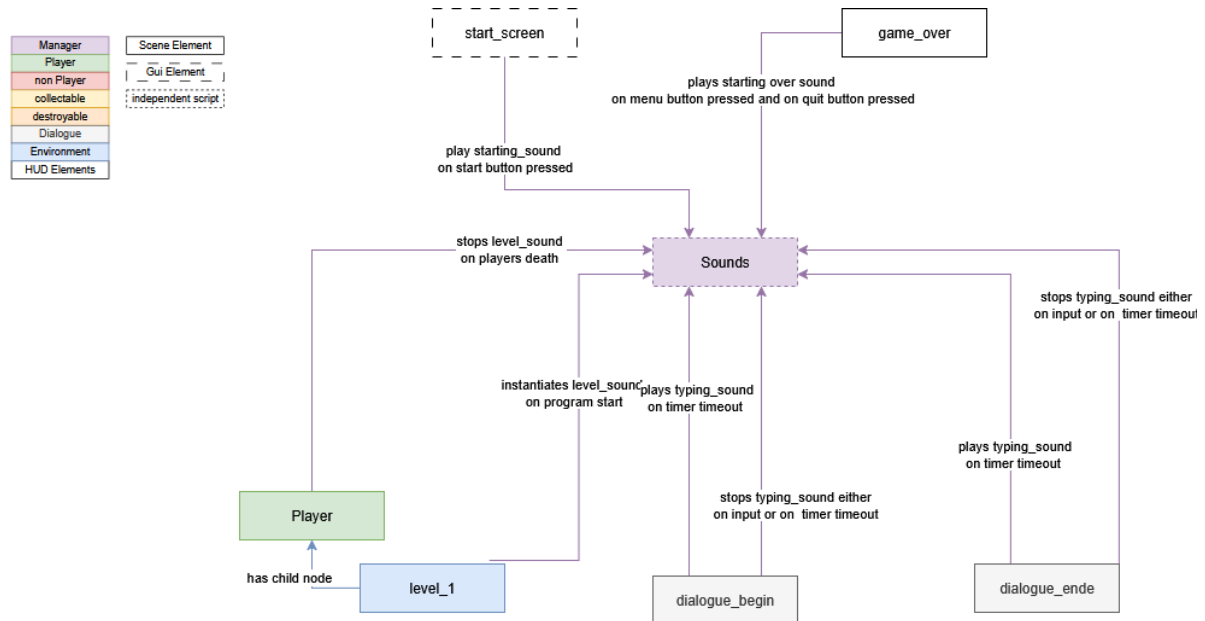
Somit kann man z.B. die Szenen überall wechseln durch den Aufruf:

`GlobalSignals.scene_controller.change_game_scene("res://Hud/startscreen.tscn")`

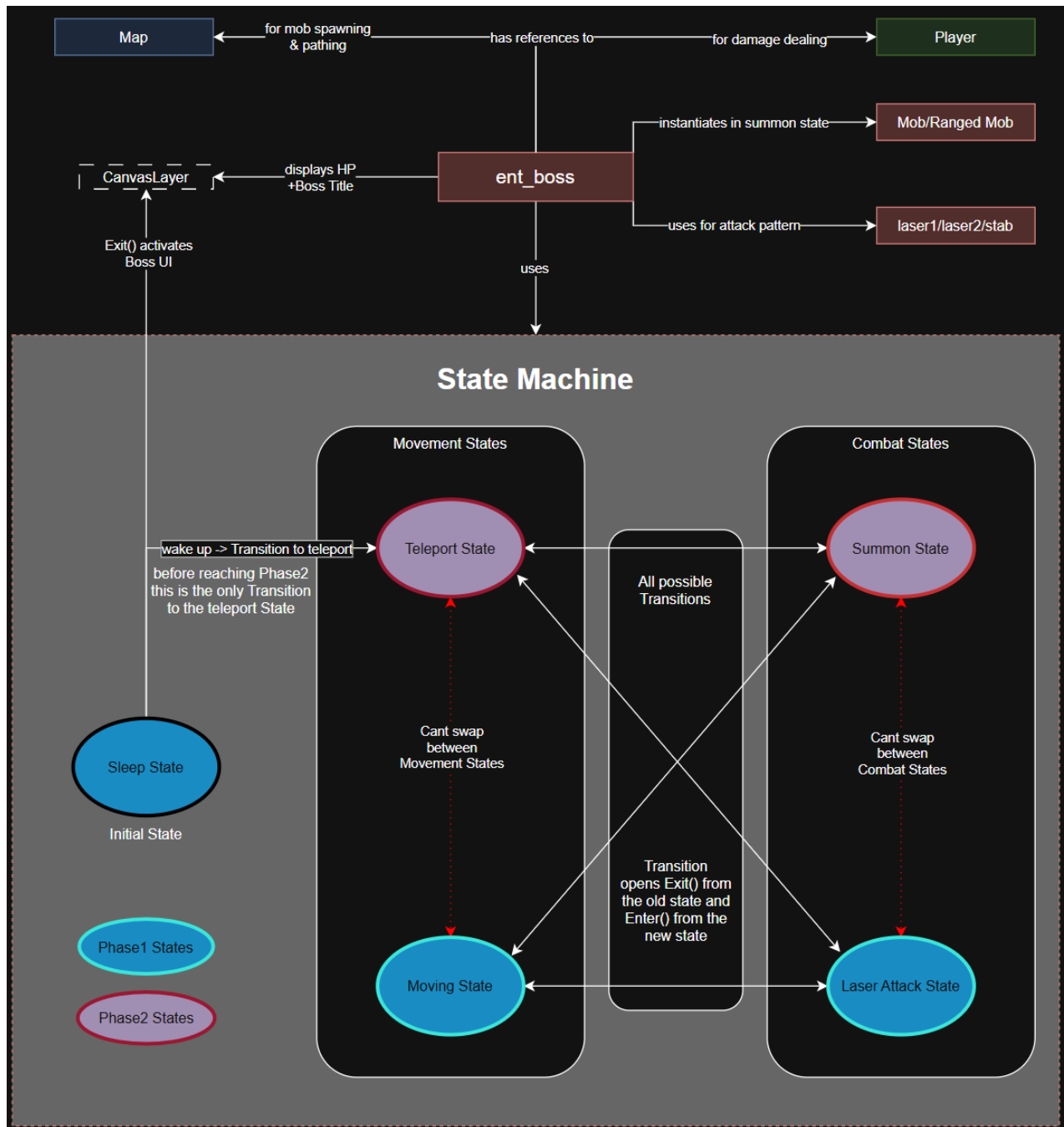


Globale Sounds Übersicht

Dieses Skript handelt global relevante Sounds, wie der sound für die Level, welcher Szenen unabhängig oder übergreifend laufen kann. Die Sounds sind im Scene_Controller unter der Sounds Node vermerkt und sind jederzeit abspielbar.



Boss State Machine Übersicht



Nachtrag

Bekannte Bugs

- Die dialogue_ende Szene wird, nachdem der Boss besiegt wurde, durch das Betreten aller Räume getriggert, nicht nur durch den PC Pool
- Während man in einem Raum steht und die letzte Ente einsammelt, wird das Dach eingeblendet durch die Temporäre deaktivieren der Player Collision Dies ist benötigt, damit Gegner einen nicht angreifen
- Der Dialog für die 2te Etage wird per Richtung getriggert; einmal wenn man die Etage betritt, einmal wenn man sie verlässt, da es zwei unterschiedliche Areas sind mit gleichem Skript

Potentielle Erweiterungen für die Zukunft

- Unterschiedliche Waffen und Spielercharaktere (Sprites bereits vorhanden)
- Dialog mit Herr Antes und Herr Müller, wenn man die Boss Ente besiegt (Sprites vorhanden)
- Destroyables auf der zweiten Etage (Platz Problem)
- Code Duplikation reduzieren (mob & mob_ranged, die ganzen Drops)
- Weitere Level wie zum Beispiel außerhalb des Gebäudes
- Lichtverhältnisse ausprobieren (z.B mit Wechsel von Tag & Nacht)
- Wandering & Follow States bei Mobs einbauen mit mini State Machines
- MeeleeAttack State erweitern für einen Meelee Boss